

Advanced Python for Scientific Computing

Michael Milligan

milligan@umn.edu

Follow along @ <https://z.umn.edu/msipython>

Spring 2026

To get the most out of this...

- Basic knowledge of: Linux, MSI, Python
- Access to campus network
 - Use [Campus VPN](#) if you are watching from off-campus
- Working MSI login
- Follow along! <https://z.umn.edu/msipython>
- Many examples will work at home!
 - [Anaconda](#) (and similar frameworks) provide easy to install Python with scientific and math libraries included
 - BinderHub link at the above URL for a scratch environment available for anyone to use online

Tutorial Outline

MORNING SESSION: 10 am - noon

- Connecting with Python @ MSI
- Getting Started with Python
- Math with Python

AFTERNOON SESSION: 1 pm - 3 pm

- Visualization and plotting
- Data processing
- Scaling up to supercomputing

When you leave today, you should be able to...

- Program interactively with **Python Notebooks**
- Understand the basics of **numpy** and **scipy**
- **Efficiently compute** with large arrays of data
- Load and save data using **files** and **web resources**
- Use **matplotlib** to visualize data
- Take advantage of supercomputing resources with **batch jobs** and **parallel computing**
- **Know where to turn** for more help with these topics!

Connecting with Python @ MSI

...

Infrastructure, tools, and background



Researcher

Needs compute resources

Secure Network

On-campus eduroam or ethernet
Off-campus via UMN VPN

MSI Access Gateway

OnDemand Portal
SSH Command Line
Other Web Portals

HPC Compute

Computation on MSI clusters
Agate partitions

Data Storage

Primary - Home+Project Dirs
2nd Tier - object storage
Scratch storage areas

Getting Connected at MSI

Recent Changes

There have been some significant changes recently that may affect your usage:

❖ **Mesabi retirement**

- The **mesabi** cluster was decommissioned
- Use **agate** for all your cluster computing needs
- The **mangi** nodes were moved to **agate**, find them there

❖ **User home directories now separate from projects**

- Your \$HOME is now in a separate /users/ space
- Your \$HOME quota is fixed and doesn't affect project space
- Find your project area (and old home directory/old files) in the usual location at /home/PROJECT/

Why Interactive Computing?

- Immediate feedback
- Prototyping workflows
 - Design your workflow for a small set of cores
 - Discover and test new tools/concepts
 - Profile, optimize and debug
- Data Visualization and Exploration
- **Python has great tools** for interactive computing

Check out our [Interactive Computing tutorial](#) for more!

Interactive Options

- **OnDemand Portal**
general interactive HPC access
- **Command line shell**
SSH or OnDemand terminal
- **Interactive batch**
srun via the command line

OnDemand Portal

<https://ondemand.msi.umn.edu>

- ❖ **OnDemand Web Portal is a new offering in 2022**
- ❖ One-stop access portal for interactive HPC services
 - Remote desktop access
 - Command line terminal
 - File browser
 - Launchers for common GUI scientific applications
 - JupyterLab server access
 - RStudio server access

OPEN

 **nDemand**

Interactive Command Line

`ssh <username>@<resource>.msi.umn.edu`

See: <https://www.msi.umn.edu/content/connecting-hpc-resources>

- SSH is the traditional first point of access to MSI resources
- Access **agate** login node
- Uses ssh program available on most personal computers

These “login nodes” are shared by many MSI users. They are NOT for heavy computational tasks. Appropriate uses include:

- ✓ Browsing and editing files
- ✓ Submitting jobs
- ✓ Querying job status and other informational commands
- ✓ Browsing software modules
- ✓ *Light* testing or software compilation

```
Microsoft Windows [Version 10.0.19041.746]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\ ssh landw020@mangi.msi.umn.edu
Password:
Last login: Wed Feb 10 15:06:20 2021 from x-10-21-31-220.vpn.umn.edu
-----
University of Minnesota Supercomputing Institute-----
-----
University of Minnesota Supercomputing Institute
Mangi
AMD EPYC Linux Cluster
-----
For assistance please contact us at https://www.msi.umn.edu/support/help.html
help@msi.umn.edu, or (612)626-0802.
-----
Home directories are snapshot protected. If you accidentally delete a file in
your home directory, type "cd .snapshot" then "ls -lt" to list the snapshots
available in order from most recent to oldest.

January 6, 2021: Slurm is now the scheduler for all nodes.
-----
landw020@ln1002 [~] %
```

Interactive Command Line

Open OnDemand

Files ▾

Jobs ▾

Clusters ▾

Interactive Apps ▾



>_ Agate Shell Access

Command line
terminal access

Command line terminal:

- “SSH” in a web browser (almost)
- Launch terminal on cluster headnode
- Roughly equivalent to e.g.
`ssh <username>@agate.msi.umn.edu`
- Also launch in specific directory via
file browser pane

>_ Open in Terminal

+ New File

New Directory

Upload

Download

Copy/Move

Delete



/ home / support / milligan /

Change directory

Copy path

Interactive Batch with ‘srun’

```
srun <resource requests> --pty bash
```

See: <https://www.msi.umn.edu/content/interactive-queue-use-srun>

- Dedicated compute resources via the scheduler
- Command line terminal running on the cluster
 - ◆ similar to a SSH session
- Supports all usual Slurm options and commands
- Best used with interactive [partitions](#)
 - ◆ have priority access for small interactive job requests
- Job terminates when shell exits or connection broken

Example:

```
srun -N 1 --ntasks-per-node=4 --mem-per-cpu=1gb -t 1:00:00 -p interactive --pty bash
```

Launch Jupyter - OnDemand

Home / My Interactive Sessions / Jupyter

Interactive Apps

Desktops

Desktop

Persistent Desktop

GUIs

ANSYS Workbench

IGV

MATLAB

IDEs

Abaqus

COMSOL Multiphysics

IDL

Mathematica

Servers

Jupyter

RStudio Server

Jupyter

This app will launch a Jupyter server using Python.

Cluster

agate

Jupyter interface

Lab

This defines the interface of Jupyter you want to start (Notebook or Lab).

Jupyter Python

MSI Python 3.8.3 / Jupyter 4.7.0

This defines the Python distribution of Jupyter you want to start.

Account

msistaff

Resources

Interactive - 2 cores, 32 GB, 64 GB local scratch

Time Limit

4 Hours

Shorter times will probably start faster

☐ I would like to receive an email when the session starts

Launch

Example launcher: Jupyter server

Options to customize most aspects of the interactive job

- cluster / partition
- group (if multiple available)
- resource requests
- launcher-specific parameters (e.g. Python environment for Jupyter servers)

Command Line Python

Run Python from an MSI command line prompt.

This is the best way to run pre-existing Python scripts.

Easy to transform command line work into batch jobs.

```
module load <python module>
```

```
python my-cool-program.py
```


Command Line IPython

IPython - the Interactive Python environment

- IPython is also the basis of the Jupyter notebook (similar syntax and features, but no graphics)
- If you are already at an MSI command-line terminal, IPython is convenient and powerful
- Most (not all) Python environments have IPython included

```
module load <python module>
```

```
ipython
```


Python modules

```
% module avail python
```

```
----- /common/software/modulefiles/spack.tcl/linux-centos7-ivybridge -----  
python/3.10.10-gcc-7.2.0-v45hj6j  python/3.10.10-gcc-8.2.0-sanagub  
python/3.10.10-gcc-8.2.0-boc3bal  python/3.10.10-gcc-13.1.0-ucftxt
```

```
----- /common/software/modulefiles/singularity-hpc -----  
python/3.12-rc/module.tcl
```

```
----- /common/software/modulefiles/migrated/hpc -----  
python/3.4                                python3/3.4
```

```
<...snip many more lines...>
```

```
python3/3.4                                python3/3.10.9_anaconda2023.03_libmamba
```

```
----- /common/software/modulefiles/bundle/views/common -----  
python-miniforge/24.11.0-0  python-rich/3.10_msi1.0
```

Most of the time this is all you need:

```
% module load python
```

What kind of Python?

Python comes in multiple versions and distributions.

- **Version?**

- Definitely use **Python 3+**
 - Many modern packages will require 3.6 or later versions
 - All 3.x versions are **forward** compatible
(code written for 3.n will work with python 3.n+1)
 - Python 2.7 - obsolete but installed for running legacy code
-
- Prefer our installed distributions, most packages you need will already be there

Where to run Python?

OnDemand - agate compute

`ondemand.msi.umn.edu`

terminal, remote desktop, or Jupyter

Batch Jobs via sbatch

Interactive Batch Jobs via srun

To get the most out of this...revisited

- Basic knowledge of: Linux, MSI, Python
- Access to campus network
 - Use [Campus VPN](#) if you are watching from off-campus
- Working MSI login
- Follow along! <https://z.umn.edu/msipython>
- Many examples will work at home!
 - [Anaconda](#) provides easy to install Python with scientific and math libraries included

No seriously. Follow along! z.umn.edu/msipython

Visit ondemand.msi.umn.edu to start your notebook.

Clone the Github repository of examples.

Or run repository through BinderHub directly from GitHub

Python Basics

Starting from “Advanced Python.ipynb”

Modules and Packages

Software libraries are organised into modules:

```
import tool
```

Tries to load:

```
./tool.py
```

```
tool.py from sys.path
```

```
tool/__init__.py from sys.path
```

```
from tool import widget
```

equals:

```
import tool
```

```
widget = tool.widget
```

Python Standard Library

Python aims to come “batteries included”

Very wide range of functionality included in the Python Standard Library

Programs using only Standard Library should run in any Python install (some caveats for older/Windows OS)

Current list is here:

<https://docs.python.org/3/library/>

Getting More Libraries

PIP Local Install

Several ways to install new modules, but 90% of the time this will work:

```
pip install --user <package name>
```

or even

```
pip install --user  
git+https://github.com/python-tool
```

- Use virtual environments for isolation if needed
- If in doubt check the package's README
- Email us for more assistance

Getting More Libraries

PIP Local Install with Jupyter

PIP can install packages usable by existing Python environment, but how to target version used by your notebook?

Use magic command from inside the notebook!

```
%pip install <package name>
```

Getting More Libraries

CONDA Environments

Anaconda environments are based on the **conda** package manager and an ecosystem of **conda** repositories

```
conda env list - lists available environments
```

To create and use a personal environment with packages of your choosing:

```
module load conda (if necessary)
```

```
conda create -n <name> python pkg1 pkg2
```

```
source activate <name>
```

```
conda install pkg3 pkg4
```

Getting More Libraries

CONDA Environments - mamba

For complex environments `conda create` or `conda install` can become painfully slow.

Recommendation: for common use-cases, use **mamba** in place of **conda** command. Usage is nearly identical.

Available via `module load conda` or install the **mamba** package into your own conda environment.

(Most of our python environments have **mamba** installed and ready to use already.)

PIP vs CONDA for installing modules

PIP

- Works great for simple modules
- Integrates with existing Python modules and managed install
- Tied to Python major version (e.g. 2.7, 3.4, 3.6)
- Can install packages directly from GitHub or local downloads
- Almost all Python software available through PyPI

CONDA

- Works great for simple or complex modules
- Creates a standalone self-contained “environment”
- Installed modules tied to specific environment that must be separately activated
- Better for software with complex or “picky” dependencies
- Most widely used software available through conda channels

Can I use new modules with Jupyter?

YES

PIP installed modules: yes, with same python version

CONDA environments: need to add new “kernelpec”

1. Activate the environment normally from terminal (use a command like "source activate ...")
2. Run the command: `jupyter kernelspec list`
3. Verify the desired environment is listed (if not, may need to install **ipykernel** package)
4. Install into Jupyter's kernel list with
python -m ipykernel install --user --name MYNAME --display-name "My Python"
5. MYNAME must be unique, and “My Python” should allow you to recognize it in the launcher

Optional: find that kernel.json file and change the “display name” to something descriptive.

High Performance Math

• • •

Numpy and Scipy

Numpy and Scipy

NumPy provides:

- Basic array and matrix data types
- Efficient implementations of low-level math operations
- A large library of high-level math functions built from efficient primitives

SciPy provides:

- A home for a wide variety of open-source mathematical and scientific algorithms
- Modules for optimization, signal processing, linear algebra, statistics, interpolation, and more

Numpy and Scipy

Let's explore these further in Jupyter...

Start from notebook/AdvPyFiles/01.NumpyArrays

more useful numpy modules...

numpy.fft – FFTs, forward/inverse, 1-D and N-D

numpy.random – generate random numbers, many distributions to choose from

numpy.matrix – special arrays that obey matrix math

numpy.polynomial – module for representing and manipulating arbitrary polynomials

SciPy expands the menu:

- [Clustering algorithms \(scipy.cluster\)](#)
- [Integration and ODEs \(scipy.integrate\)](#)
- [Interpolation \(scipy.interpolate\)](#)
- [Input and output \(scipy.io\)](#)
- [Linear algebra \(scipy.linalg\)](#)
- [Multi-dimensional image processing \(scipy.ndimage\)](#)
- [Optimization and root finding \(scipy.optimize\)](#)
- [Signal processing \(scipy.signal\)](#)
- [Sparse matrices \(scipy.sparse\)](#)
- [Spatial algorithms and data structures \(scipy.spatial\)](#)
- [Special functions \(scipy.special\)](#)
- [Statistical functions \(scipy.stats\)](#)
- And then some...

SciPy is also fast!

- Most SciPy routines use NumPy for fast low-level math operations
- Some SciPy routines use highly optimized external libraries
- E.g. `scipy.linalg` links to BLAS, LAPACK or MKL behind the scenes
- For even faster Python, stay tuned for more options later in this session



Data Visualization

• • •

Matplotlib friends and alternatives

Plotting made easy

- **Matplotlib** provides publication-quality 2-D plotting (with some 3D capabilities too)
 - Display in window or output to PDF, SVG, PNG, etc
 - Implemented as modular object-oriented system
 - IPython: enable with `%matplotlib`
- **Pylab** provides a Matlab-ish interactive plotting interface to Matplotlib
 - Usually accessed via `import matplotlib.pyplot as plt`
 - Defaults to popping up plots in a separate window
 - In notebook: use `%matplotlib inline` to enable plots in the browser window

More plotting resources

These are built on **matplotlib** and will interoperate well

- **Seaborn** specializes in statistical plots, provides similar styles and capabilities to popular R plotting packages
- **Pandas** has built-in plotting with some convenience features for working from DataFrames
- **Basemap, cartopy** etc for GIS graphics needs

There are other plotting mechanisms out there too:

- **Bokeh** focuses on interactive web visualizations
- **Plotly** integrates with plot.ly for interactive web
- **ggplot** aims to replicate ggplot2 from R
- **Mayavi** high performance 3D modelling/volumetric app

Data Processing

...

Loading saving and processing datasets

Data on Disk

Chances are you want to **load** and **save** data!

- **numpy** and **scipy.io** offer a variety of generic options
- **pandas** can read and write many tabular formats
- several specialized options for large/complex data

Text files: portable

- Very common for smaller data sets:
 - simple columns of numbers
 - human readable in a pinch
- `numpy.loadtxt()` – simple interface, good defaults
- `numpy.genfromtxt()` – more complex, handles unusual formatting, comments, missing values, etc

Text files: portable

More common options

- CSV files
 - Pandas: fast, good defaults
 - csv module: standard library option
- JSON
 - Pandas: fast, good defaults (again!)
 - json module: standard library

Binary formats

- **Binary data** is much more scalable
- Smaller files on disk
- Faster to load and save
- May be necessary to exchange data with other software
- Stick to portable (machine-independent) formats

Binary formats

- **NumPy native format (.npy)**

- `numpy.load()` and `numpy.save()`
- Or use `numpy.savez()` to store many arrays in compressed .npz
- Fast, portable, but mostly only supported by Python

- **scipy.io.matlab** – support for Matlab (.mat)

- `scipy.io.loadmat()` and `scipy.io.savemat()`

- **scipy.io.idl** – read (no save) IDL .sav files

- `scipy.io.readsav()`

Binary formats

(recommended)

- HDF5 - compact table-oriented datasets
 - pytables, pandas, h5py
- NetCDF4 - similar to HDF5
 - prefer HDF5 for new projects
- Parquet - gaining traction in data science
 - fastparquet, pandas, several others
- Pickle - python's native serialization format
 - e.g. for complex data structures, data-with-code, etc
- Special mention: Apache Arrow
 - an in-memory binary format increasingly used behind the scenes by data science tools (including Pandas 2.0)

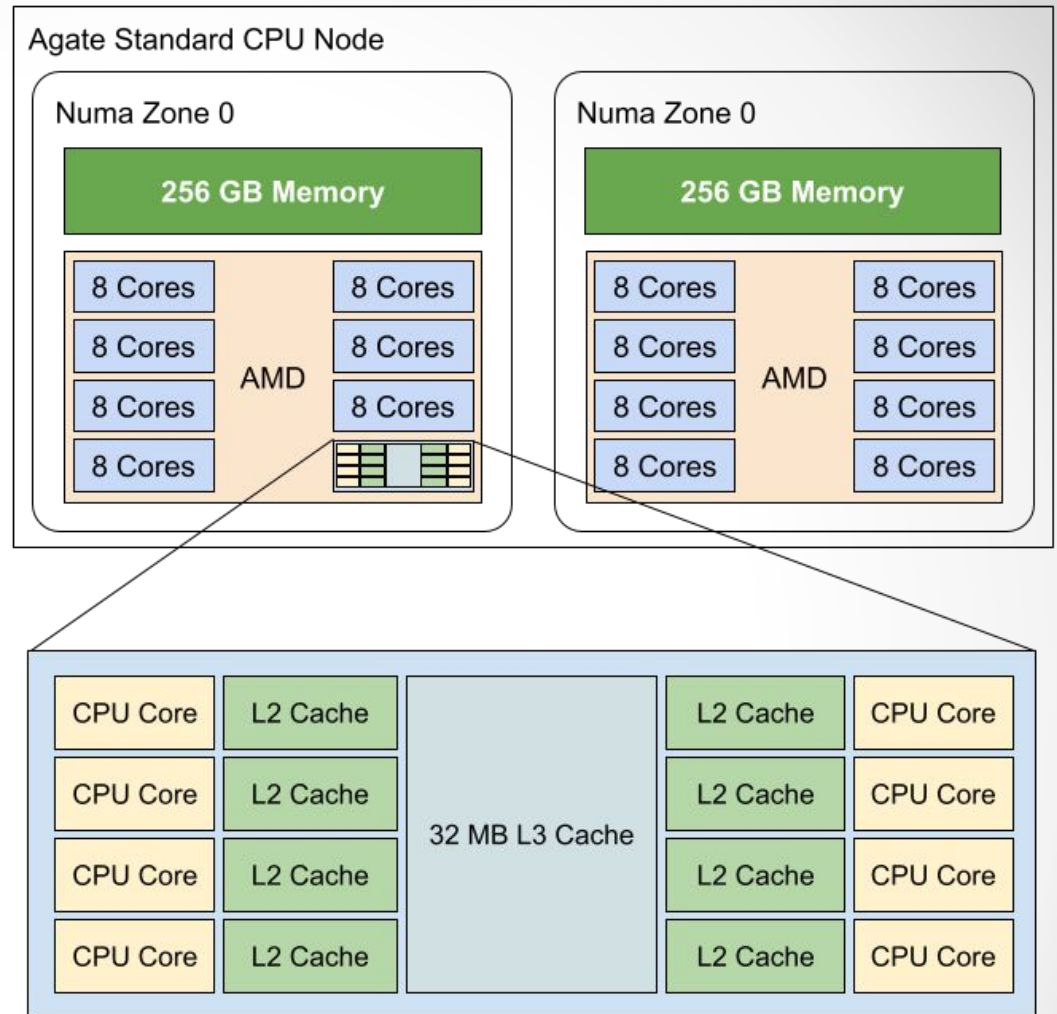
Scaling up to HPC

...

Acceleration and parallelism

Standard Agate CPU node

- 128 CPU cores
- 512 GB of memory
- 850 GB local SSD



Acceleration: going faster

Acceleration means using the CPU efficiently

- **numpy**: use pre-compiled efficient primitives
- **Cython**: compile functions with a python-like syntax
- **numba**: just-in-time optimization engine

Proceed to:

```
notebook/AdvPyFiles/  
Acceleration-cython-numba.ipynb
```

Parallelism: going wider

Modern CPUs are multi-core

- parallelism is usually beneficial
- can be combined with acceleration techniques
- higher complexity due to “splitting up” work

Proceed to:

`notebook/AdvPyFiles/MultiprocessingExample.ipynb`

Further Reading and Resources

- Official Python Documentation
<https://docs.python.org/3/>
- Numpy/Scipy Docs Hub
<https://docs.scipy.org/doc/>
- IPython Cookbook
<http://ipython-books.github.io/>
- Jupyter Notebook “Gallery” - actually a huge hub of resources and examples!
<https://github.com/jupyter/jupyter/wiki/A-gallery-of-interesting-Jupyter-Notebooks>
- Contact us!
help@msi.umn.edu or <https://msi.umn.edu/helpdesk>

Supplemental Material

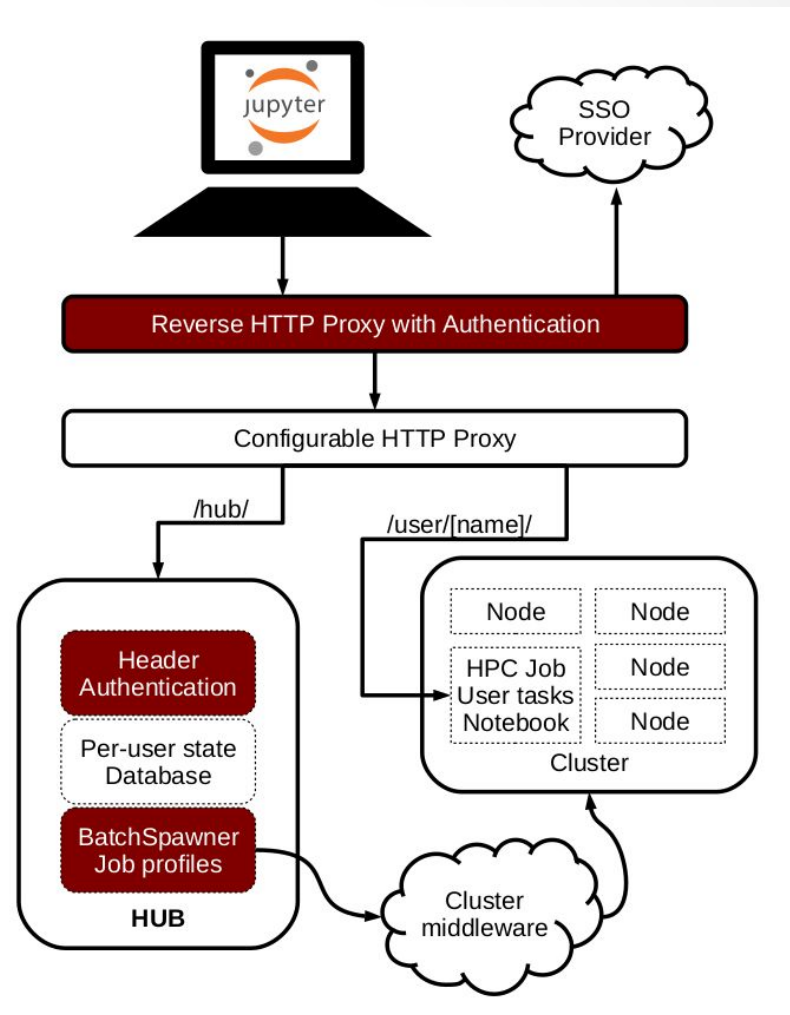
...

Notebooks = Jobs

Main compute on cluster nodes

- default “interactive” queues
- larger memory options
- longer runtime options

Jupyterhub and OnDemand work similarly



Additional Documentation

- [Overview of IPython's architecture for parallel and distributed computing.](#)
- [Detailed discussion of IPython cluster controller and engines.](#)
- [Discussion of IPython magic commands](#)
- [Official IPython documentation](#)

Batch vs. Interactive Computing

	Batch (qsub)	Interactive SSH (qsub -I)	Interactive Desktop (Linux)	Interactive Desktop (Windows)
Wall clock limit	696 hours	696 hours ^{1,2}	24 hours	48 hours
Requires SUs	✓	✓ ³	✗ ⁴	✗
Memory limit	1 TB	1 TB	16 GB	61 GB
Core Limit	8640	8640	4	8
Software Modules	400+	400+	400+	50+
GPUs	✓	✓	✓	✓
GUIs	✗	✓	✓	✓

1. Don't be a jerk 2. Larger requests receive lower priority 3. Resource dependent 4. Subject to change